

TAREA 5 · PROGRAMACIÓN II

Implementación del Diagrama de Clases en el Sistema de Ventas

Java Swing · Patrón MVC · SQLite con JDBC · Formulario contenedor MDI

Josue Zetino

Agosto de 2026

Repositorio <https://github.com/jzetinob/sistema-ventas>

Objetivo

Trasladar el diagrama de clases del sistema de ventas a código Java real, manteniendo la arquitectura MVC, la base de datos SQLite y el formulario contenedor MDI de las tareas anteriores.

Cada clase, atributo, método y relación del diagrama quedó implementado y funcionando

6 clases

Persona, Cliente, Empleado, Producto, Factura y DetalleFactura llevadas a archivos .java.

4 relaciones

Herencia, dos asociaciones y una composición, cada una visible en el código.

2 cambios en la BD

Tabla empleados y columna existencia, creadas por migración automática.

1 catálogo nuevo

Catálogo de Empleados funcionando desde el menú de la aplicación.

Clases implementadas: del diagrama al código

Cada clase del diagrama tiene su archivo .java dentro del paquete modelo.

Persona

«abstract»

id, nombre, nit, telefono y el método concreto mostrarInformacion()

modelo/Persona.java

Cliente

hereda de Persona

extends Persona más el atributo direccion con getDireccion()

modelo/Cliente.java

Empleado

hereda de Persona

extends Persona con codigoEmpleado y puesto; sobrescribe mostrarInformacion()

modelo/Empleado.java

Producto

clase

idProducto, nombre, precio, existencia y hayExistencia(cantidad)

modelo/Producto.java

DetalleFactura

composición

producto: Producto, cantidad, precioUnitario y calcularSubtotal()

modelo/FacturaDetalle.java

Factura

clase

idFactura, fecha: LocalDate, cliente: Cliente, detalles: List<DetalleFactura>, agregarDetalle() y calcularTotal()

modelo/Factura.java

Relaciones del diagrama en el código

Las cuatro relaciones del UML tienen su equivalente en Java y en el esquema de la base de datos.

1

Herencia

`Persona → Cliente / Empleado`

Ambas clases usan `extends Persona`. `Empleado` redefine `mostrarInformacion()`, lo que demuestra polimorfismo; `Cliente` conserva el método de la clase padre.

```
class Empleado extends Persona
```

2

Asociación 1..*

`Cliente ↔ Factura`

Una factura tiene un solo cliente; un cliente puede participar en muchas facturas. En la base de datos la relación se guarda con la columna `cliente_id`.

```
cliente_id en facturas
```

3

Composición

`Factura → DetalleFactura`

Los detalles pertenecen exclusivamente a la factura: se agregan y eliminan solo a través de ella, y mueren con ella.

```
ON DELETE CASCADE
```

4

Asociación

`DetalleFactura ↔ Producto`

Cada detalle apunta a un producto, pero no lo posee: el mismo producto puede aparecer en muchos detalles y facturas.

```
producto_id en factura_detalles
```

Cambios en la base de datos y funcionalidad nueva

El diagrama obligó a ampliar el esquema existente sin perder los datos de tareas anteriores.

Tabla empleados

Se crea automáticamente en ConexionBD al arrancar la aplicación, con el código de empleado como campo único.

```
CREATE TABLE IF NOT EXISTS empleados
```

Columna existencia

Se agrega con ALTER TABLE automático sobre bases de datos ya creadas, de modo que la migración no pierde información.

```
ALTER TABLE productos ADD existencia
```

Lo que el usuario ve en la aplicación

Catálogo de Empleados

El menú Catálogos → Empleados abre un CRUD completo con persistencia en SQLite mediante EmpleadoDAO y EmpleadoDAOSQLite, validación de código único y botones Guardar, Actualizar, Eliminar y Limpiar.

Existencia de productos

El CRUD de Productos muestra y mantiene la columna Existencia. Al agregar un producto a la factura se valida hayExistencia() y la cantidad se resta al guardar.

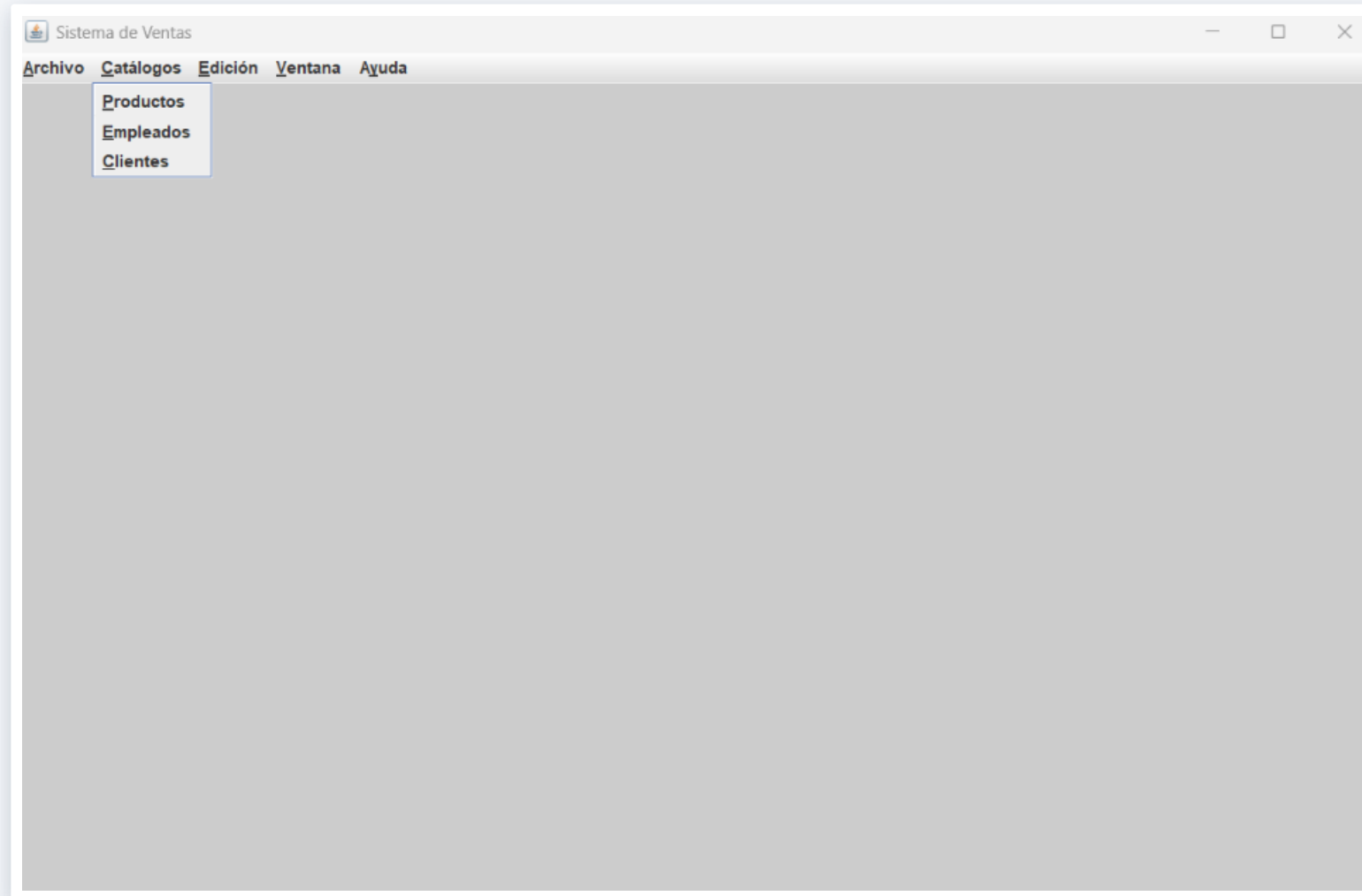
02

La aplicación en ejecución

Ocho capturas del sistema funcionando: catálogos, diagrama, facturación e impresión.

Menú Catálogos → Empleados

La nueva opción se integra al menú existente de la ventana principal MDI.



Catálogo de Empleados

CRUD completo con código, nombre, NIT, teléfono y puesto, persistido en SQLite.

Sistema de Ventas

Catálogo de Empleados

Datos del Empleado

Código: **Guardar**

Nombre: **Actualizar**

NIT: **Eliminar**

Teléfono: **Limpiar**

Puesto:

No.	Código	Nombre	NIT	Teléfono	Puesto
2	EMP-001	Juan Perez	123456789	5555-1010	Cajero
3	EMP-002	Maria Lopez	9876543210	5555-2020	Vendedora

Catálogo de Productos con existencia

La columna Existencia se agregó al formulario y a la tabla sin perder los productos anteriores.

Sistema de Ventas

Archivo Catálogos Edición Ventana Ayuda

Catálogo de Productos

Datos del Producto

Código: Guardar

Nombre: Actualizar

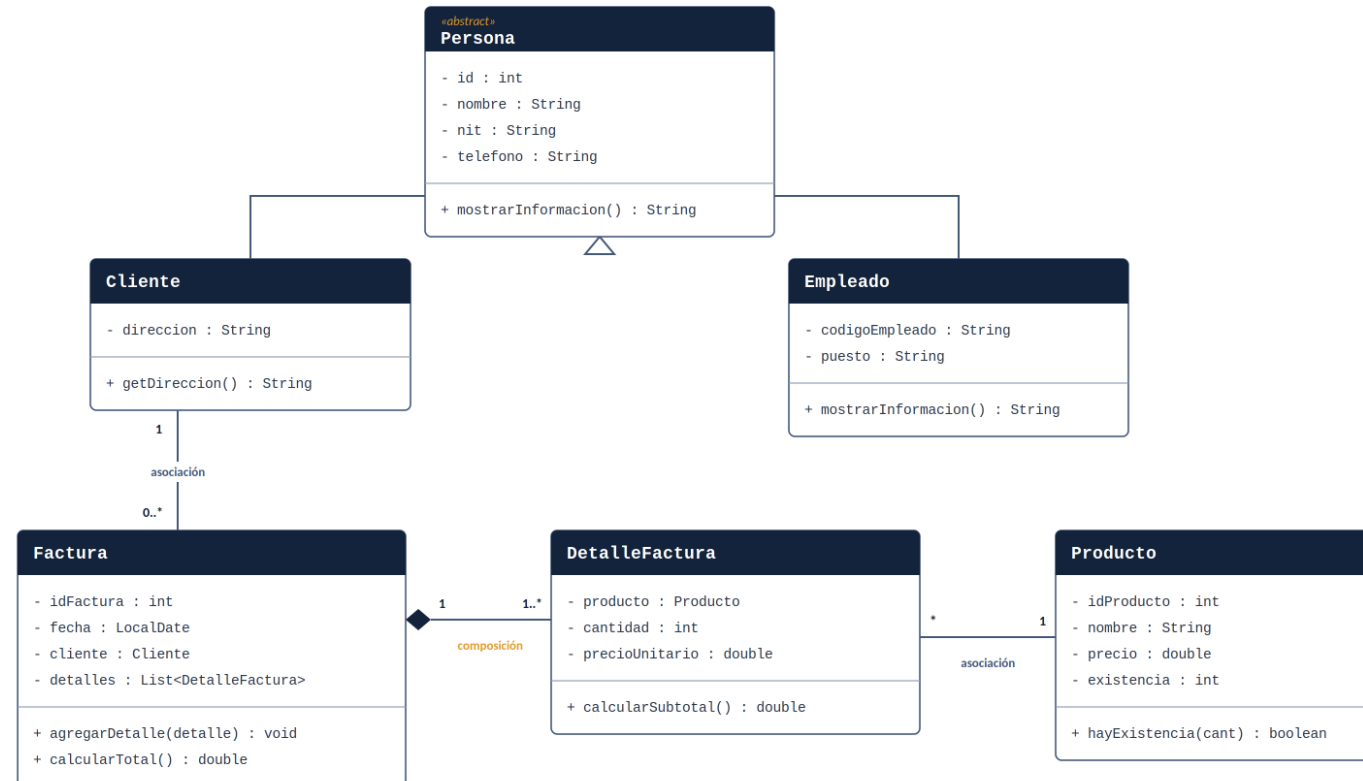
Precio: Eliminar

Existencia: Limpiar

No.	Código	Nombre	Precio	Existencia
1	900	laptop	5350.00	0
2	901	Mouse	55.00	0
3	902	Teclado	95.00	0
4	903	cargador 12v	150.00	0
6	904	mini pc	2500.00	0

Diagrama de clases del sistema

Persona como clase abstracta, sus dos herederas, y las relaciones con Producto, Factura y DetalleFactura.



● herencia → extends

● composición → ON DELETE CASCADE

● asociación → llave foránea

Formulario de facturación

Los datos del cliente y la tabla de productos alimentan la composición Factura → DetalleFactura.

Sistema de Ventas

Formulario de Facturación

Datos del Cliente

Cliente: Fecha:

NIT: No. Factura:

Cliente del catálogo:

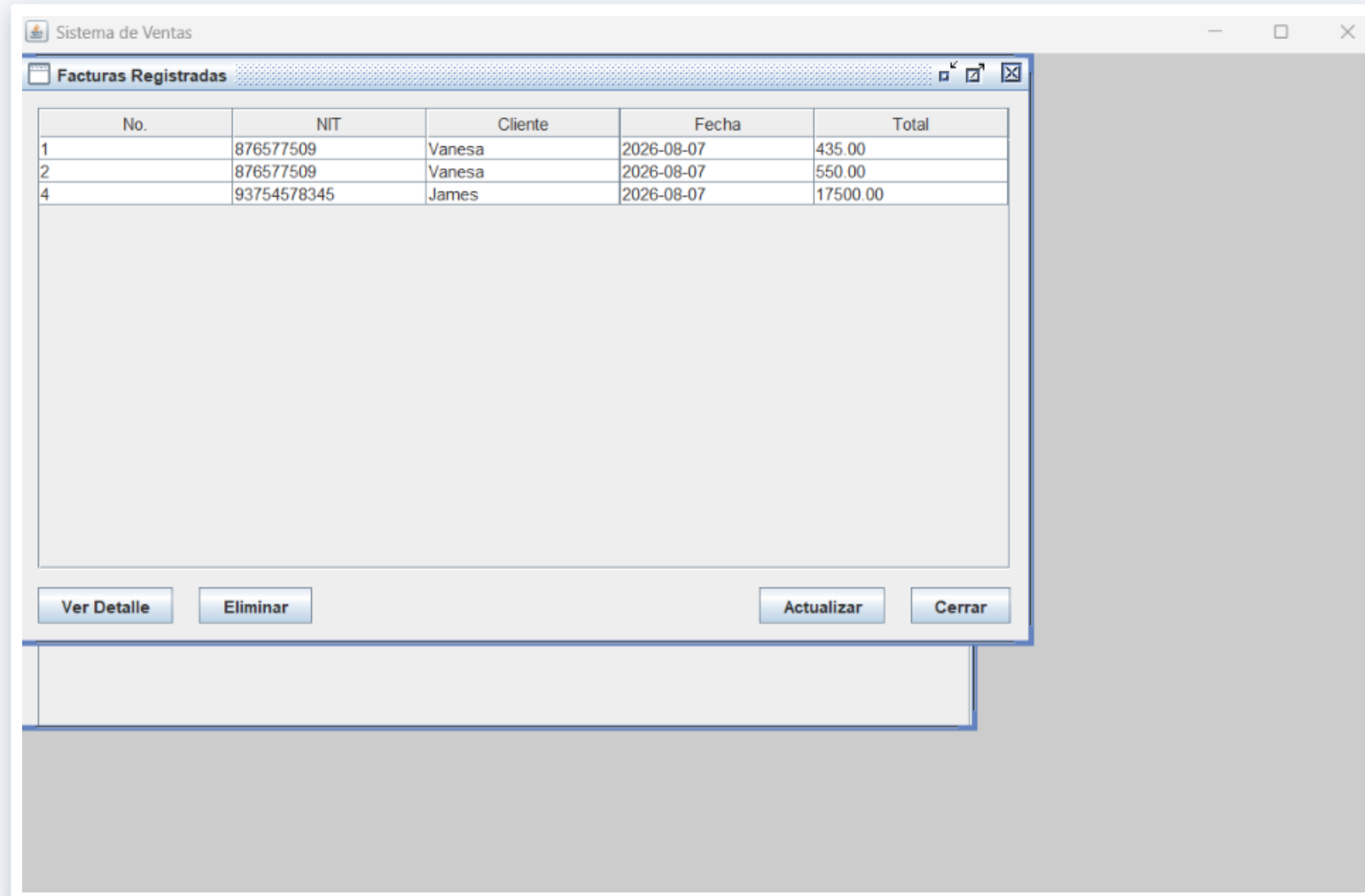
Agregar Producto

Producto: Cantidad: Precio:

Producto	Cantidad	Precio	Subtotal
----------	----------	--------	----------

Lista de facturas registradas

Cada fila corresponde a una Factura con su cliente asociado, recuperada desde la base de datos.



The screenshot shows a software window titled 'Sistema de Ventas'. Inside, there is a sub-window titled 'Facturas Registradas'. This sub-window contains a table with the following data:

No.	NIT	Cliente	Fecha	Total
1	876577509	Vanesa	2026-08-07	435.00
2	876577509	Vanesa	2026-08-07	550.00
4	93754578345	James	2026-08-07	17500.00

Below the table, there are four buttons: 'Ver Detalle', 'Eliminar', 'Actualizar', and 'Cerrar'. The 'Actualizar' button is disabled.

Detalle de una factura

Los detalles de la factura FAC-0001 con sus subtotales y el total calculado por calcularTotal().

Sistema de Ventas

Detalle de Factura

No. Factura: FAC-0001 NIT: 876577509 Cliente: Vanesa

Fecha: 2026-08-07 Total: 435.00

Producto	Cantidad	Precio	Subtotal
902 - Teclado	4	95.0	380.0
901 - Mouse	1	55.0	55.0

Imprimir Cerrar

Cerrar

Vista previa del ticket

El mismo objeto Factura se recorre para armar el ticket antes de enviarlo a la impresora.

SISTEMA DE VENTAS

No. Factura: FAC-0001
Fecha: 2026-08-07
Cliente: Vanesa
NIT: 876577509

PRODUCTO	CANT	PRECIO	SUBTOTAL
902 - Teclado	4	95.00	380.00
901 - Mouse	1	55.00	55.00

TOTAL: 435.00

Gracias por su compra

Imprimir

Cerrar

Verificación

Comprobaciones realizadas sobre el proyecto ya implementado.

Compilación

Compilación completa del proyecto con javac (JDK 25), sin errores.

Prueba de humo automatizada

Cinco comprobaciones ejecutadas sobre la aplicación, todas con resultado [OK].

OK

Creación de tablas

Incluye la tabla empleados y la columna existencia.

OK

CRUD de EmpleadoDAO

Alta, consulta, modificación y baja de empleados.

OK

hayExistencia()

Validación de existencia antes de agregar a la factura.

OK

Composición de Factura

calcularTotal() devuelve 151.00 sobre los detalles cargados.

OK

Menú Catálogos → Empleados

El formulario abre correctamente desde la ventana MDI.

Conclusiones

El diagrama de clases no quedó como dibujo: cada elemento se programó, se persistió y se usa.

1

Polimorfismo comprobado

Empleado sobrescribe mostrarInformacion() mientras Cliente conserva el método de Persona.

2

Composición y asociación funcionando

Los detalles viven y mueren con su factura, mientras los productos se comparten entre muchas.

3

Todo accesible desde la aplicación

El catálogo de empleados y la existencia de productos se usan desde la ventana principal MDI.

Repositorio público

<https://github.com/jzetinob/sistema-ventas>